#SKILL - 7 REST API - CRUD Operations using ResponseEntity Prerequisites:

• General Idea on Spring Boot MVC Architecture

• Knowledge of Service and Controller layers A university application needs backend services to manage course details such as courseId, title, duration, and fee. The admin should be able to add, update, delete, and view course information. The system must return proper HTTP status codes and structured responses for reliable communication between client and server. Your task is to implement REST endpoints using ResponseEntity to perform complete CRUD operations on course data.

Tasks: 1. Create a Course entity with fields: courseId, title, duration, fee.

2. Create a Service layer that performs full CRUD operations.

3. Implement REST endpoints using @PostMapping, @PutMapping, @DeleteMapping, and @GetMapping.

4. Use ResponseEntity to return: a. Success messages b. Error messages c. Status codes such as OK, CREATED, NOT_FOUND, BAD_REQUEST

5. Add a search endpoint /courses/search/{title} to filter courses by title.

6. Test all endpoints (valid and invalid cases) using Postman.

7. Push the Spring Boot project to GitHub. Title: Course Management REST API using Spring Boot

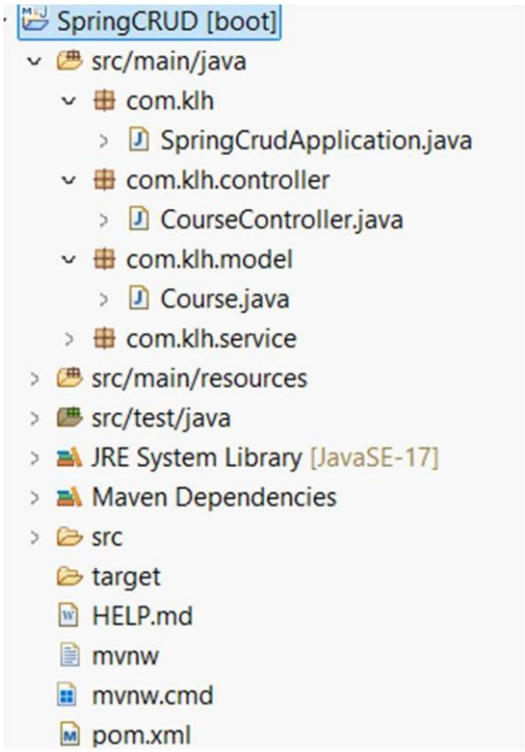
Objective: To implement CRUD operations on Course data using Spring Boot REST APIs and Response Entity.

Required Dependencies

1. **Spring Web-** For building REST APIs
   (Provides @RestController, @GetMapping, ResponseEntity, etc.)

2. **Spring Boot DevTools -** For auto-restart during development.

Create the Spring starter project with the following structure:



Course.Java

```
package com.klh.model;

public class Course {

   private int courseId;

   private String title; private

   String duration; private

   double fee; public

   Course() {}

   public Course(int courseId, String title, String duration, double fee) {

      this.courseId = courseId;

      this.title = title; this.duration =
```

```
        duration; this.fee = fee;
    }
    public int getCourseId() {
        return courseId;
    }
    public void setCourseId(int courseId) {
        this.courseId = courseId;
    }
    public String getTitle() {
        return title;
    }
    public void setTitle(String title) { this.title =
        title;
    }
    public String getDuration() {
        return duration;
    }
    public void setDuration(String duration) {
        this.duration = duration;
    }
    public double getFee() {
        return fee;
    }
    public void setFee(double fee) { this.fee =
        fee;
    }
}
```

CourseController.java

```
package com.klh.controller;

import com.klh.model.Course;

import com.klh.service.CourseService;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController

@RequestMapping("/courses")

public class CourseController {

   private final CourseService courseService;

   public CourseController(CourseService courseService) {

      this.courseService = courseService;

   }

   // CREATING THE COURSE

   @PostMapping

   public ResponseEntity<?> addCourse(@RequestBody Course course) {

      if (course.getCourseId() <= 0) {

         return new ResponseEntity<>("Invalid Course ID", HttpStatus.BAD_REQUEST);

      }

      return new ResponseEntity<>(courseService.addCourse(course),
HttpStatus.CREATED);

   }

   // READING ALL THE COURSES

   @GetMapping
```

```
    public ResponseEntity<List<Course>> getAllCourses() {
        return new ResponseEntity<>(courseService.getAllCourses(), HttpStatus.OK);
    }
    // READING THE COURSE BY ID
    @GetMapping("/{id}")
    public ResponseEntity<?> getCourseById(@PathVariable int id) {
        Course course = courseService.getCourseById(id);
        if (course == null) {
            return new ResponseEntity<>("Course not found", HttpStatus.NOT_FOUND);
        }
        return new ResponseEntity<>(course, HttpStatus.OK);
    }
    // UPDATING THE COURSE BY ID
    @PutMapping("/{id}")
    public ResponseEntity<?> updateCourse(@PathVariable int id, @RequestBody Course
course) {
        Course updated = courseService.updateCourse(id, course);
        if (updated == null) {
            return new ResponseEntity<>("Course not found", HttpStatus.NOT_FOUND);
        }
        return new ResponseEntity<>(updated, HttpStatus.OK);
    }
    // DELETING THE COURSE
    @DeleteMapping("/{id}")
    public ResponseEntity<String> deleteCourse(@PathVariable int id) {
        boolean deleted = courseService.deleteCourse(id);
```

```
        if (!deleted) {
            return new ResponseEntity<>("Course not found", HttpStatus.NOT_FOUND);
        }
        return new ResponseEntity<>("Course deleted successfully", HttpStatus.OK);
    }
    // SEARCH BY TITLE OF THE COURSE
    @GetMapping("/search/{title}")
    public ResponseEntity<List<Course>> searchCourse(@PathVariable String title) {
        return new ResponseEntity<>(courseService.searchByTitle(title), HttpStatus.OK);
    }
}
```

CourseService.java

```
package com.klh.service;

import com.klh.model.Course;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class CourseService {

    private final List<Course> courses = new ArrayList<>();

    public Course addCourse(Course course) {
        courses.add(course);
        return course;
    }
```

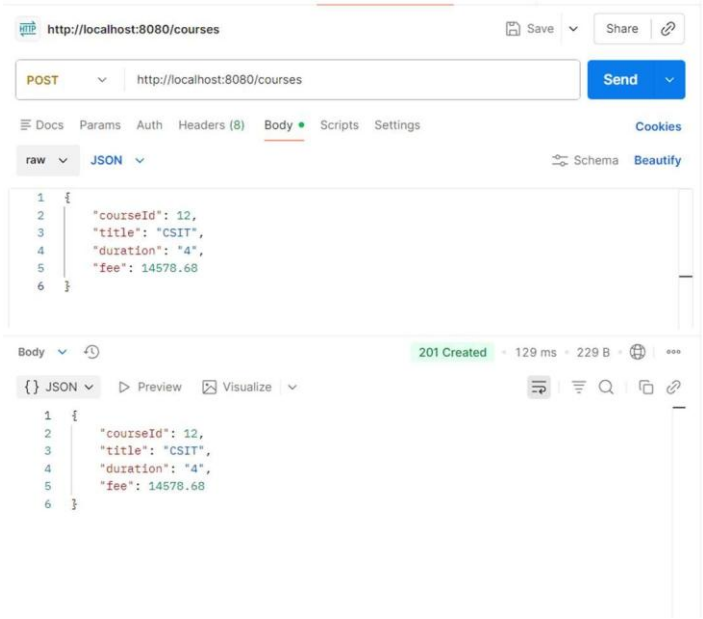
```java
public List<Course> getAllCourses() {
    return courses;
}

public Course getCourseById(int id) {
    return courses.stream()
            .filter(c -> c.getCourseId() == id)
            .findFirst()
            .orElse(null);
}

public Course updateCourse(int id, Course newCourse) {
    Course existing = getCourseById(id);
    if (existing == null) return null;

    existing.setTitle(newCourse.getTitle());
    existing.setDuration(newCourse.getDuration());
    existing.setFee(newCourse.getFee());
    return existing;
}

public boolean deleteCourse(int id) {
    return courses.removeIf(c -> c.getCourseId() == id);
}

public List<Course> searchByTitle(String title) {
    List<Course> result = new ArrayList<>();
    for (Course c : courses) {
        if (c.getTitle().toLowerCase().contains(title.toLowerCase())) {
            result.add(c);
        }
    }
    return result;
}
```

}

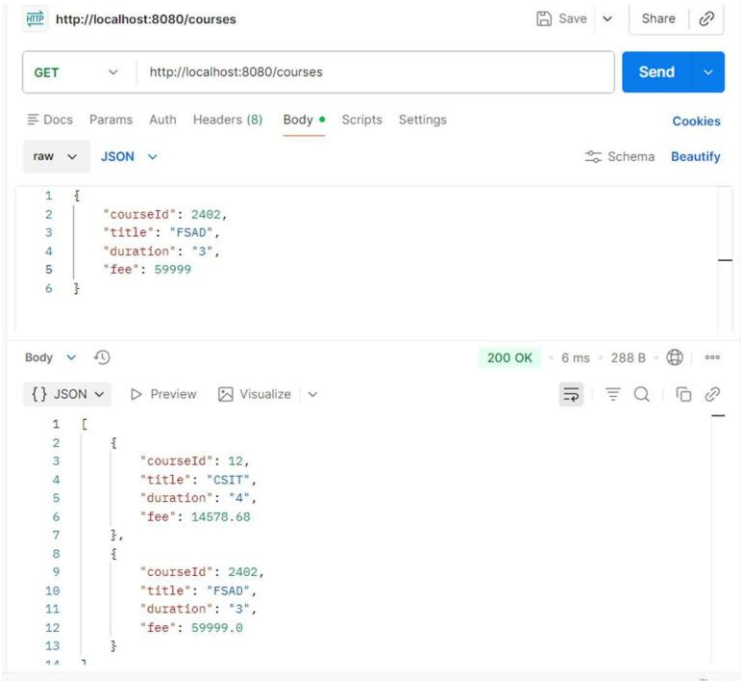
Now Run the application as SpringStarter App and Test the Endpoints in the Postman Desktop App:

To add the Course -	POST	/courses
To view the courses added -	GET	/courses
To update the course by id -	PUT	/courses/{id}
To delete the course by id -	DELETE	/courses/{id}
To search for the course by title -	GET	/courses/search/{title}

1. Add Course –



2. View All the Courses added –



3. Update Course –

Overview | PUT http://localhost:8080/cc | No environment

http://localhost:8080/courses/5 | Save | Share

PUT | http://localhost:8080/courses/5 | Send

Docs | Params | Auth | Headers (8) | Body | Scripts | Settings | Cookies

raw | JSON | Schema | Beautify

```
1 {
2     "courseId":5,
3     "title": "Front End Development Frameworks",
4     "duration": "6",
5     "fee": 25000
6 }
```

Body | 200 OK | 14 ms | 250 B

{} JSON | Preview | Visualize

```
1 {
2     "courseId": 5,
3     "title": "Front End Development Frameworks",
4     "duration": "6",
5     "fee": 25000.0
6 }
```

http://localhost:8080/courses | Save | Share

GET | http://localhost:8080/courses | Send

Docs | Params | Auth | Headers (8) | Body | Scripts | Settings | Cookies

raw | JSON | Schema | Beautify

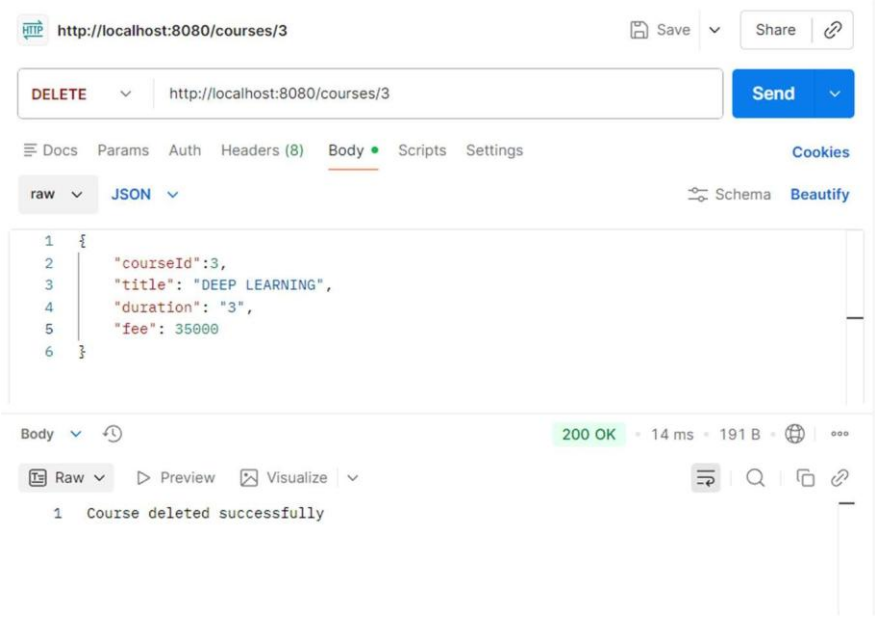
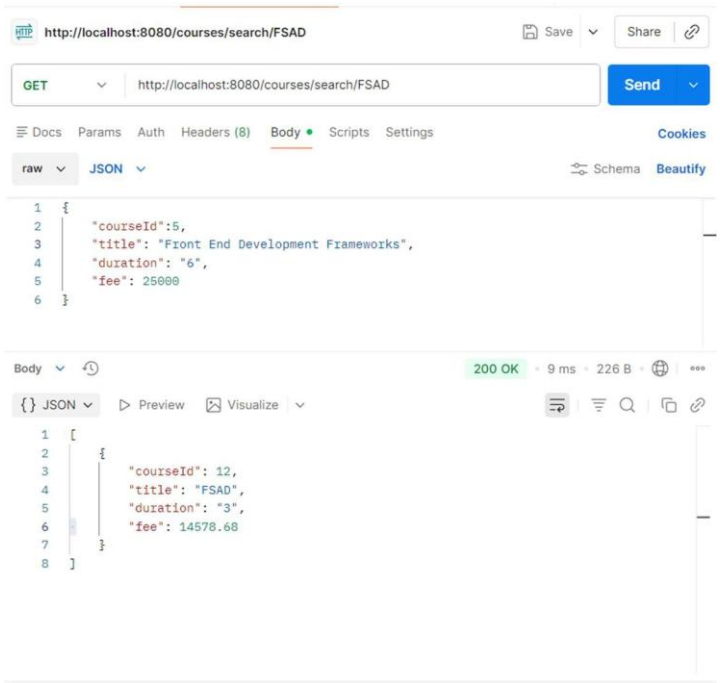
```
1 {
2     "courseId":5,
3     "title": "Front End Development Frameworks",
4     "duration": "6",
5     "fee": 25000
6 }
```

Body | 200 OK | 9 ms | 313 B

{} JSON | Preview | Visualize

```
1  [
2      {
3          "courseId": 12,
4          "title": "FSAD",
5          "duration": "3",
6          "fee": 14578.68
7      },
8      {
9          "courseId": 5,
10         "title": "Front End Development Frameworks",
11         "duration": "6",
12         "fee": 25000.0
13     }
```

4. Delete Course –



5. Search Course –